

Taller 2: Redes Neuronales

Juan Sebastian Salina Moscote - 122301

Carlos Fernando Vargas Sierra - 122041

Haider Camilo Villalba Santos - 122301

Universidad ECCI

Sistemas Avanzados de la Producción

Docente: Fredy Alexander Orjuela Lopez

Bogotá, Colombia

Introducción

El presente taller tiene como propósito aplicar de manera práctica diversos métodos de analítica predictiva y aprendizaje automático, abarcando desde modelos estadísticos tradicionales hasta técnicas modernas de inteligencia artificial. A lo largo de su desarrollo se estudian herramientas como la regresión lineal múltiple, la regularización Ridge, el descenso de gradiente, los árboles de decisión, las redes neuronales multicapa y los modelos de clasificación para mantenimiento predictivo, permitiendo analizar tanto problemas de regresión como de clasificación.

Cada fase del taller busca fortalecer la comprensión conceptual y técnica de estos modelos, profundizando en aspectos clave como la interpretación de coeficientes, la optimización numérica, el ajuste de hiperparámetros, la evaluación mediante métricas de desempeño y la relación entre el comportamiento del modelo y su impacto en escenarios reales de negocio. De esta manera, no solo se analiza el desempeño estadístico de los algoritmos, sino también su utilidad práctica en la toma de decisiones empresariales.

En conjunto, este trabajo permite evidenciar cómo la correcta selección, ajuste e interpretación de modelos predictivos constituye un elemento esencial para transformar datos en conocimiento accionable, generando valor estratégico para las organizaciones en contextos de marketing, optimización operativa y mantenimiento predictivo.

1. Fase 1: Desarrollo Analítico: Regularización Ridge.

Desarrollo Analítico: Regularización de Ridge

Función dada:

$$J(\beta) = \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \sum_{j=1}^p \beta_j^2$$

- Representación vectorial:

$$Y = X\beta + \varepsilon$$

Donde:

$$Y \in R^{n \times 1}$$

$$X \in R^{n \times p}$$

$$\beta \in R^{p \times 1}$$

Definimos la función del costo como:

$$J(\beta) = (Y - X\beta)^T (Y - X\beta) + \lambda \beta^T \beta$$

$$\sum \beta_j^2 = \beta^T \beta$$

- Expansión de la forma cuadrática:

$$(Y - X\beta)^T (Y - X\beta)$$

$$= Y^T Y - Y^T X\beta - \beta^T X^T Y + \beta^T X^T X\beta$$

Como $Y^T X\beta$ es escalar:

$$Y^T X \beta = \beta^T X^T Y$$

$$= Y^T Y - 2\beta^T X^T Y + \beta^T X^T X \beta$$

$$J(\beta) = Y^T Y - 2\beta^T X^T Y + \beta^T X^T X \beta + \lambda \beta^T \beta$$

Agrupamos:

$$J(\beta) = Y^T Y - 2\beta^T X^T Y + \beta^T (X^T X + \lambda I) \beta$$

- Condición de primer orden:

Derivamos e igualamos a 0:

$$\frac{\partial J}{\partial \beta} = -2X^T Y + 2(X^T X + \lambda I)\beta$$

$$-2X^T Y + 2(X^T X + \lambda I)\beta = 0$$

Dividimos en 2:

$$(X^T X + \lambda I)\beta = X^T Y$$

- Solución cerrada:

$$\widehat{\beta}_{Ridge} = (X^T X + \lambda I)^{-1} X^T Y$$

Respuesta a las preguntas:

- **Sobre la Invertibilidad:** ¿Por qué la suma de λI garantiza que la matriz resultante sea invertible incluso si $X^T X$ es singular (no invertible)?

RTA: La matriz $X^T X$ puede no ser invertible cuando existen variables predictoras contienen información similar o redundante, cuando el número de predictores es mayor que el número de observaciones ($p > n$), o cuando algunas variables están altamente correlacionadas entre sí. En estos casos, la matriz puede ser singular o casi singular, lo que impide calcular la solución de mínimos cuadrados ordinarios.

Al agregar el término de regularización λI , se obtiene la matriz:

$$X^T X + \lambda I$$

Este término se añade λ a los valores propios de la matriz $X^T X$. Si algunos valores propios eran muy pequeños o incluso cero, al sumar λ se garantiza que todos sean estrictamente positivos ($v_i + \lambda > 0$).

Como resultado:

- Todos los valores propios se vuelven positivos.
- La matriz se convierte en definida positiva.
- Por lo tanto, es invertible.

En consecuencia, la regularización Ridge no solo penaliza los coeficientes, sino que también estabiliza el problema numéricamente y asegura que la solución exista incluso cuando OLS falla.

- **Efecto del Parámetro λ :** ¿Qué sucede con el modelo y sus coeficientes cuando $\lambda \rightarrow 0$?
¿Y qué ocurre cuando $\lambda \rightarrow \infty$? Explique en términos de la complejidad del modelo (Sesgo vs. Varianza).

RTA: El parámetro λ controla el nivel de penalización sobre los coeficientes del modelo.

Cuando $\lambda \rightarrow 0$

El estimador Ridge converge al estimador de Mínimos Cuadrados Ordinarios:

$$\widehat{\beta}_{Ridge} = (X^T X + \lambda I)^{-1} X^T Y$$

Esto implica que prácticamente no hay penalización. En este caso:

- El modelo tiene baja penalización.
- Puede presentar mayor varianza.
- Existe riesgo de sobreajuste (overfitting).

Es decir, el modelo puede ajustarse demasiado a los datos de entrenamiento.

Cuando $\lambda \rightarrow \infty$

El término de penalización domina la función de costo y los coeficientes tienden a cero:

$$\widehat{\beta}_{Ridge} \rightarrow 0$$

Esto significa que:

- Los coeficientes se reducen considerablemente.
- El modelo se vuelve muy simple.
- Aumenta el sesgo (bias).
- Disminuye la varianza.
- Existe riesgo de sobreajuste (underfitting).

El modelo pierde capacidad para capturar patrones complejos en los datos.

Interpretación en términos de Sesgo-Varianza

El parámetro λ regula el equilibrio entre sesgo y varianza:

- Cuando λ es pequeño:
 - Bajo sesgo.
 - Alta varianza.
 - Mayor riesgo de sobreajuste.
- Cuando λ es grande:
 - Alto sesgo.
 - Baja varianza.
 - Mayor riesgo de sobreajuste.

Ridge busca encontrar un valor óptimo de λ que equilibre el sesgo y la varianza, permitiendo obtener un modelo que generalice mejor a nuevos datos.

2. Fase 2: Optimización Numérica y Descenso de Gradiente

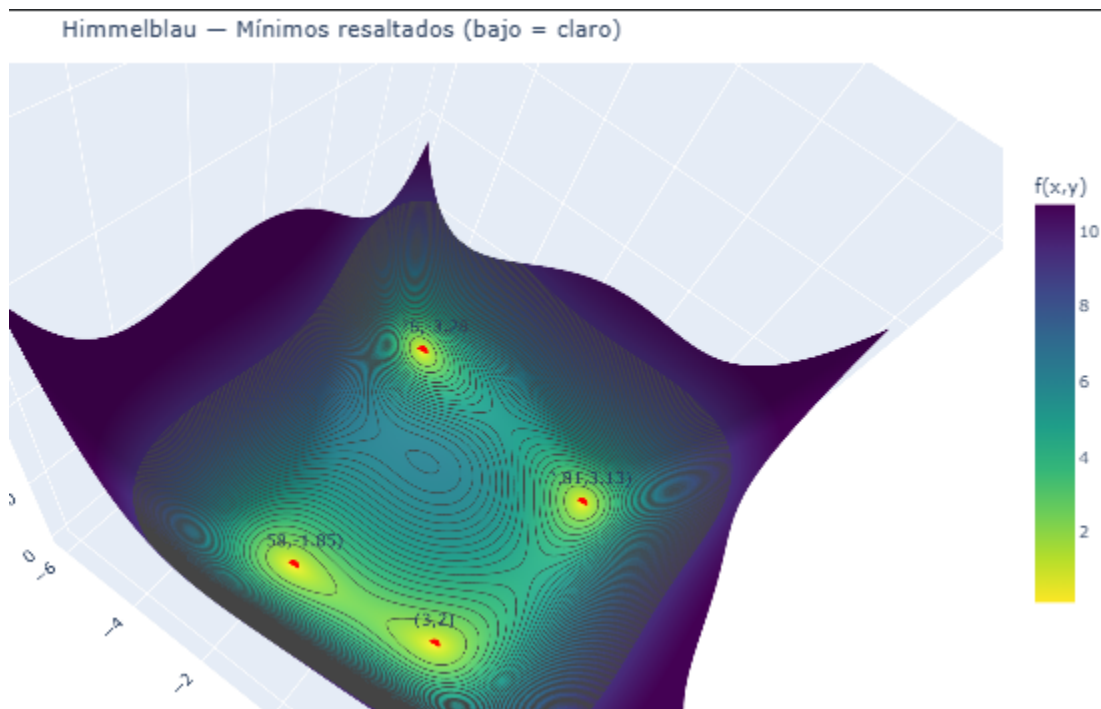
Desarrollo Optimización Numérica y Descenso de Gradiente

La función que estás minimizando es la función de Himmelblau:

$$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

Esta función tiene múltiples mínimos locales, lo que permite analizar la dependencia del punto inicial.

- Impacto de la tasa de aprendizaje (η)



Configuración Estándar (0.0001)

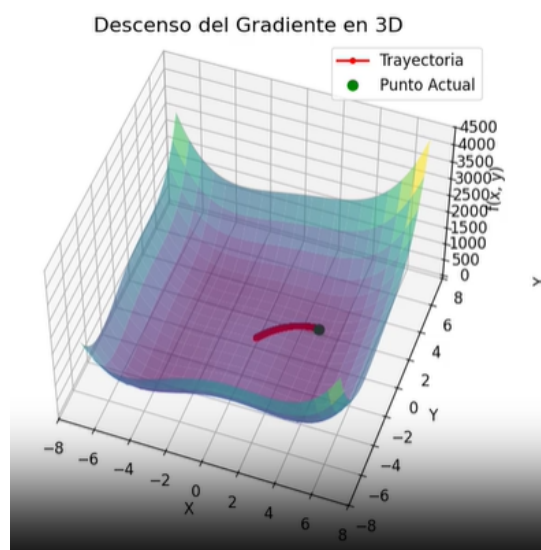


Imagen 1. learning_rate, Estándar descenso del gradiente en 3D. Converge lentamente hacia un mínimo cercano.

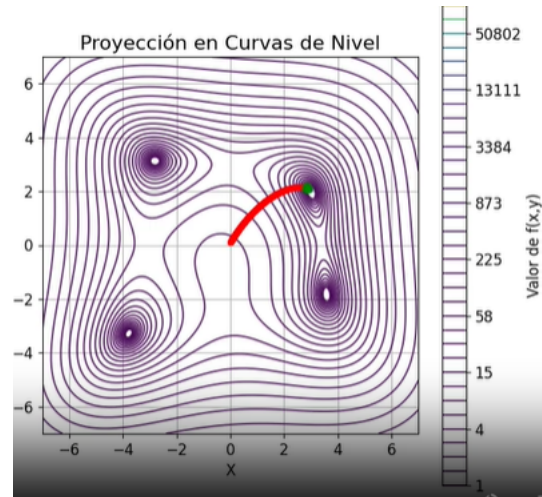


Imagen 2. learning_rate, Estándar proyección en curvas de nivel en 3D. Converge lentamente hacia un mínimo cercano.

Análisis: Converge lentamente pero alcanza un mínimo cercano.

Configuración Muy baja ($1e-7$)

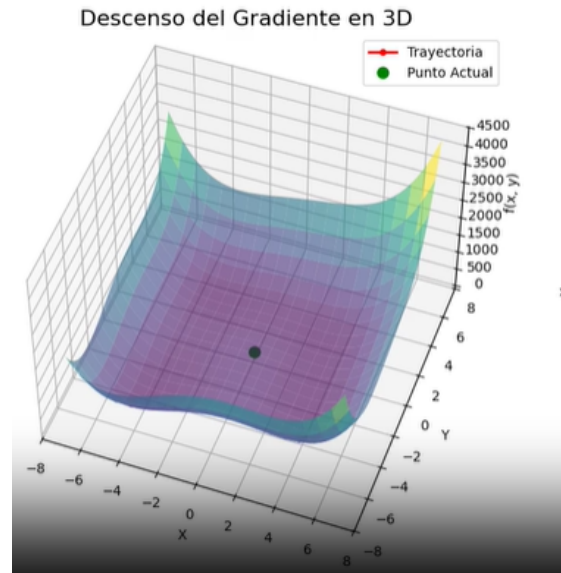


Imagen 3. `learning_rate`, Muy baja descenso del gradiente en 3D. El algoritmo apenas se mueve del punto inicial.

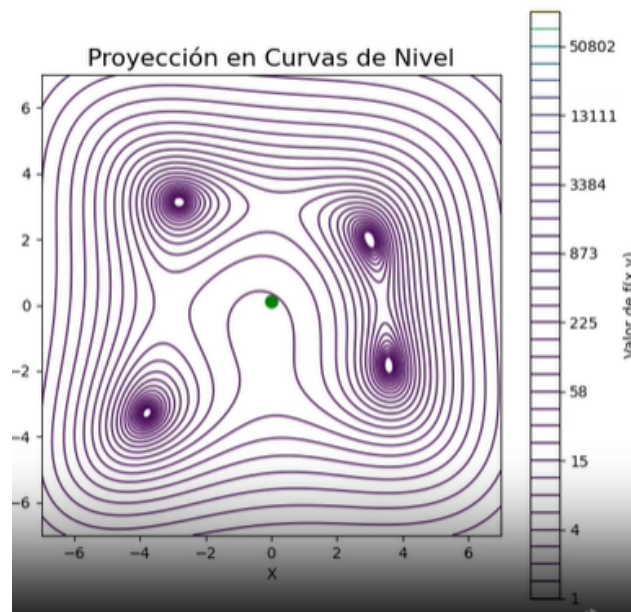


Imagen 4. `learning_rate`, muy baja proyección en curvas de nivel en 3D. El algoritmo apenas se mueve del punto inicial.

Análisis:

- El algoritmo avanza muy lentamente.
- Apenas se mueve del punto inicial.
- Con 1000 iteraciones puede no llegar al mínimo.

Cuando η es demasiado pequeño, el descenso es muy lento. El algoritmo puede no alcanzar el mínimo dentro del número de iteraciones establecido, aunque teóricamente sí convergería si se permitieran más iteraciones.

Configuración Alta (0.01)

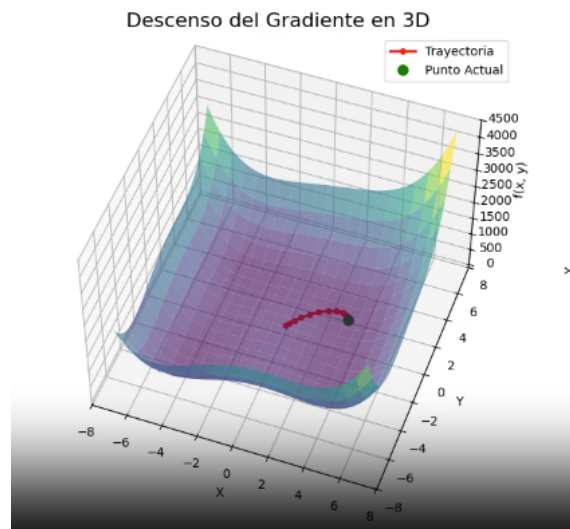


Imagen 5. learning_rate, alta descenso del gradiente en 3D. Se observan oscilaciones alrededor del mínimo.

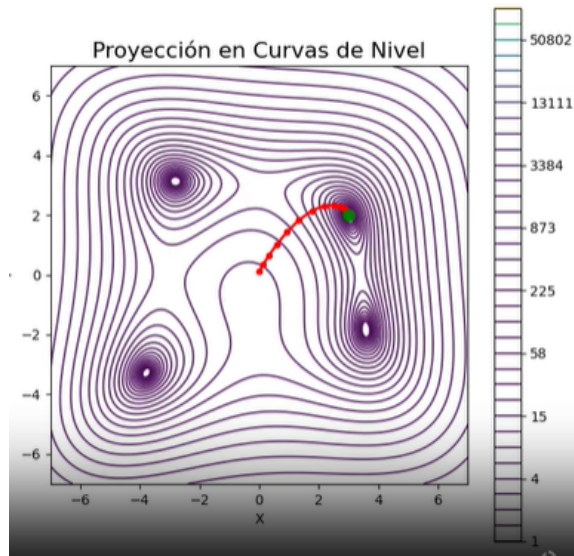


Imagen 6. `learning_rate`, alta proyección en curvas de nivel en 3D. Se observan oscilaciones alrededor del mínimo.

Análisis: Puede oscilar o divergir (overshooting).

Configuración Muy Alta (0.1)

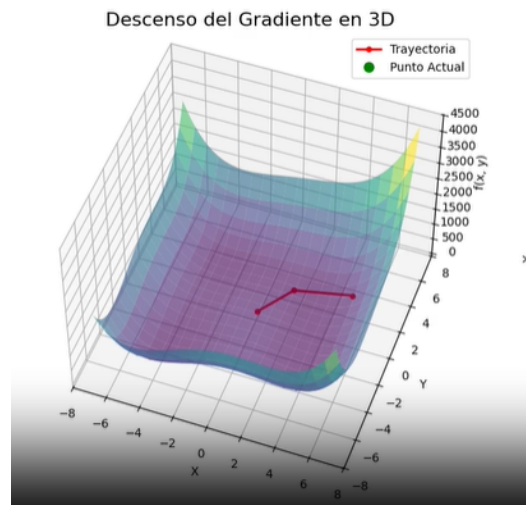


Imagen 7. `learning_rate`, muy alta descenso del gradiente en 3D. El algoritmo diverge, los puntos se alejan del mínimo.

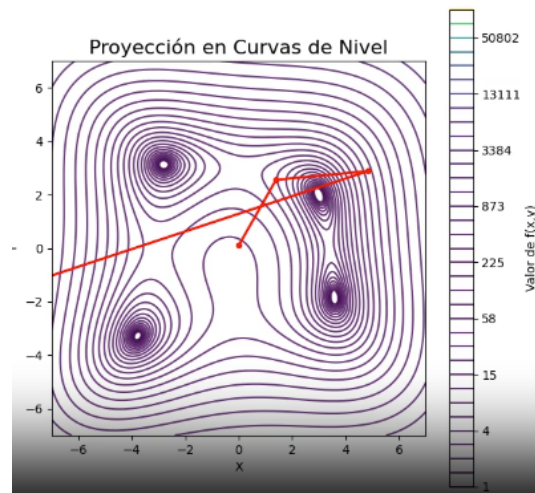


Imagen 8. `learning_rate`, muy alta proyección en curvas de nivel en 3D. El algoritmo diverge, los puntos se alejan del mínimo.

Análisis:

- El algoritmo da saltos muy grandes.
- Se pasa del mínimo.
- Puede oscilar.
- Puede divergir (irse hacia infinito).
- No convergen.

Cuando la tasa de aprendizaje es muy alta, el algoritmo sobrepasa el mínimo en cada iteración. Esto genera oscilaciones alrededor del punto óptimo o incluso divergencia, fenómeno conocido como **overshooting**.

- Dependencia del punto inicial

start_point = [6.0, -6.0]

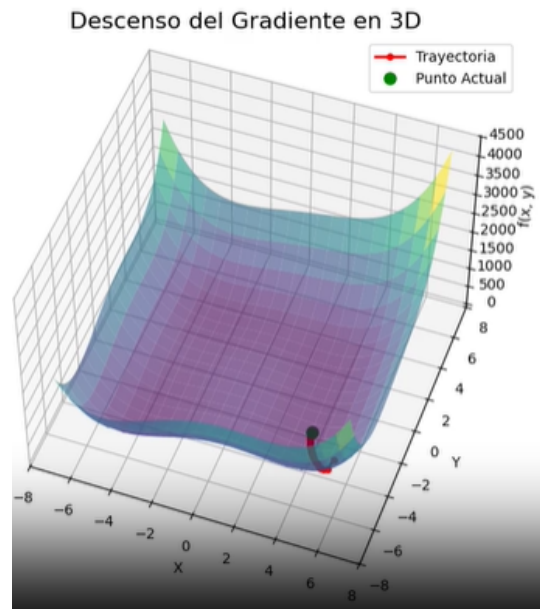


Imagen 9. start_point [6.0, -6.0], descenso del gradiente en 3D.

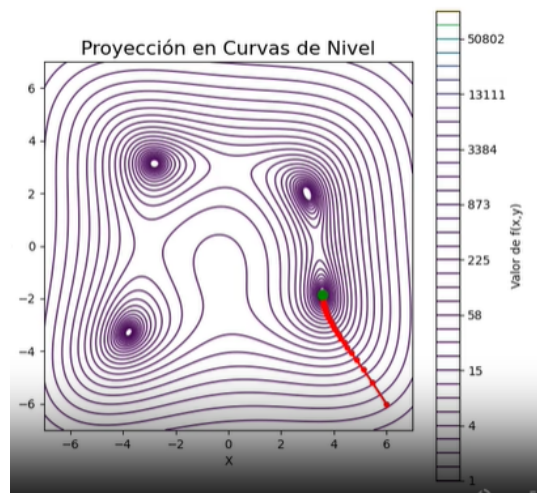


Imagen 10. start_point [6.0, -6.0], proyección en curvas de nivel en 3D.

Gráfico mostrando a qué mínimo converge (debe ser diferente al anterior)

`start_point = [-5.0, 5.0]`

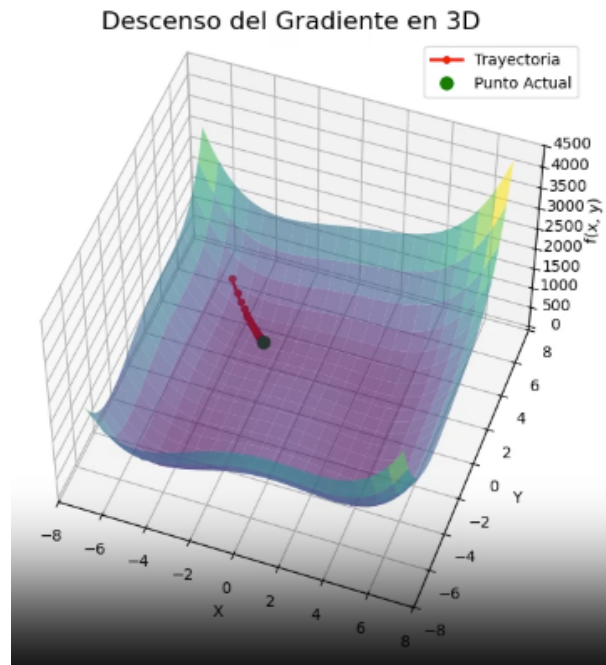


Imagen 11. `start_point [-5.0, 5.0]`, descenso del gradiente en 3D.

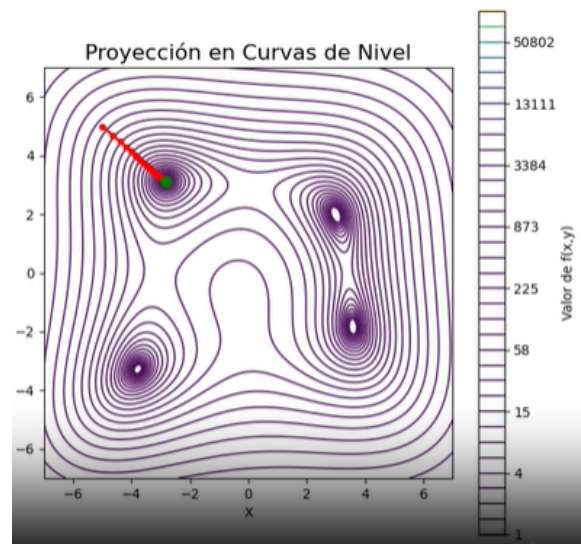


Imagen 12. start_point [-5.0, 5.0], proyección en curvas de nivel en 3D.

Gráfico mostrando a qué mínimo converge

start_point = [3.0, 2.0]

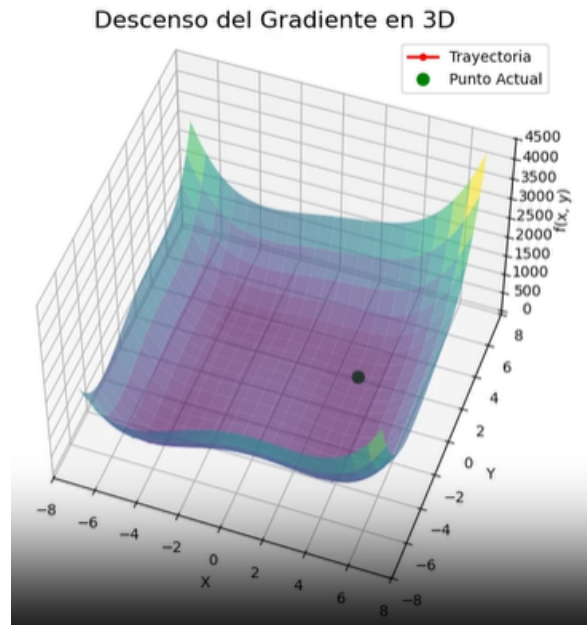


Imagen 13. start_point [3, 2], descenso del gradiente en 3D.

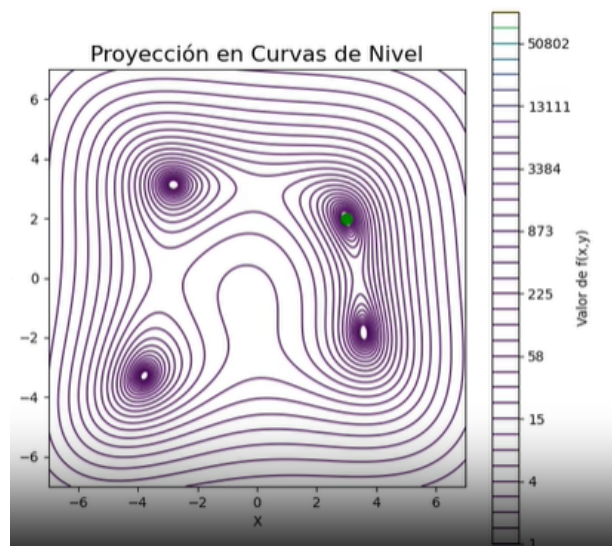


Imagen 14. start_point [3, 2], proyección en curvas de nivel en 3D.

Gráfico mostrando convergencia rápida (ya está casi en un mínimo)

La función de Himmelblau tiene varios mínimos locales:

- (3, 2)
- (-2.8, 3.1)
- (-3.8, -3.3)
- (3.6, -1.8)

Dependiendo del punto inicial, el algoritmo cae en distintos mínimos. El descenso de gradiente converge al mínimo más cercano al punto inicial. Debido a la presencia de múltiples mínimos locales, el resultado final depende del punto de partida. Esto demuestra que el algoritmo no garantiza encontrar el mínimo global.

Preguntas Fase 2

El fenómeno de “Overshooting”: Explique físicamente qué ocurre cuando el paso ($\eta \cdot \nabla f$) es más grande que la distancia al mínimo. ¿Cómo se visualiza esto en el gráfico de curvas de nivel?

Rta: El fenómeno de *overshooting* ocurre cuando la tasa de aprendizaje (η) es demasiado grande. En lugar de acercarse de manera progresiva al mínimo de la función, el algoritmo da pasos excesivamente amplios que lo llevan a sobrepasar el punto óptimo en cada iteración. Como consecuencia, pueden generar oscilaciones alrededor del mínimo o incluso divergencia del algoritmo.

Matemáticamente, la actualización en descenso de gradiente está dada por:

$$w_{t+1} = w_t - \eta \nabla f(w_t)$$

Si η es muy grande, el término $\eta \nabla f(w_t)$ produce desplazamientos demasiado amplios en el espacio de búsqueda. Esto impide que el algoritmo estabilice su trayectoria y puede alejarlo del mínimo en lugar de acercarlo.

Para evitar el overshooting se pueden aplicar las siguientes estrategias:

- Reducir la tasa de aprendizaje.
- Utilizar tasas de aprendizaje adaptativas.
- Implementar optimizadores más avanzados como Adam o RMSprop, que ajustan dinámicamente el tamaño del paso.

Eficiencia Computacional: Si el algoritmo se detiene en la iteración 1000, pero aún se encuentra lejos del mínimo, ¿qué estrategia es mejor: aumentar el número de iteraciones o ajustar la tasa de aprendizaje? Justifique su respuesta.

Rta: Si después de 1000 iteraciones el algoritmo no ha alcanzado el mínimo, es necesario analizar el comportamiento de la trayectoria antes de decidir qué ajuste realizar. Si el movimiento es muy lento, puede ser conveniente aumentar ligeramente la tasa de aprendizaje. Si el algoritmo presenta oscilaciones, es necesario reducir η . Si la convergencia es estable pero lenta, puede ser adecuado aumentar el número de iteraciones. Por lo tanto, no siempre aumentar el número de iteraciones es la solución correcta. En muchos casos, el problema radica en una tasa de aprendizaje mal ajustada.

Generalización: En un problema de ingeniería con cientos de variables (como la optimización de una cadena de suministro), ¿por qué es peligroso confiar en un único punto de inicio aleatorio?

Rta: En aplicaciones reales de ingeniería, como la optimización de redes eléctricas, sistemas logísticos o redes de suministro, la presencia de múltiples mínimos locales puede representar un desafío importante.

Si el algoritmo se inicia desde un único punto, puede converger hacia una solución subóptima en lugar de encontrar la mejor configuración global. Esto puede afectar costos, eficiencia operativa y estabilidad del sistema.

Para mitigar este problema, se utilizan estrategias como:

- Múltiples inicializaciones desde distintos puntos.
- Métodos estocásticos.
- Técnicas de optimización global como Simulated Annealing.

Estas estrategias aumentan la probabilidad de encontrar soluciones más cercanas al óptimo global.

3. Fase 3: Redes Neuronales (MLP) y Ajuste de Hiperparametros

Desarrollo Redes Neuronales (MLP) y Ajuste de Hiperparametros

Métricas del modelo base

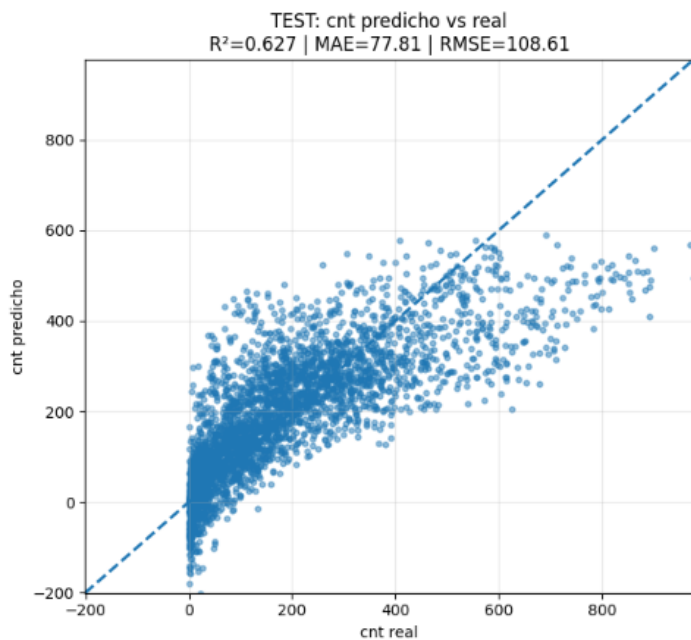


Imagen 15. TEST: cnt predicho vs real

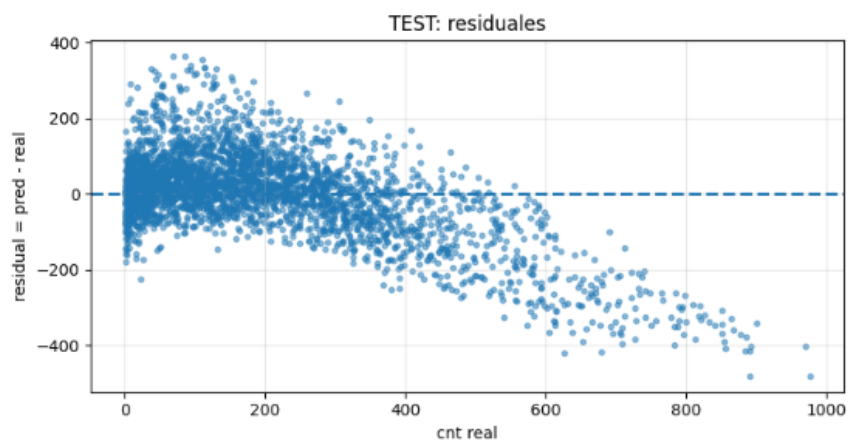


Imagen 16. TEST: Residuales

== MLPRegressor ==

Train -> MAE: 47.086 | RMSE: 68.547 | R2: 0.858

Test -> MAE: 47.175 | RMSE: 69.458 | R2: 0.848

El modelo muestra un rendimiento muy similar en entrenamiento y prueba, lo que indica que no presenta sobreajuste y generaliza bien. El MAE es de aproximadamente 47, por lo que las predicciones se desvían en promedio en esa magnitud. El RMSE es cercano a 69, con valores muy similares entre entrenamiento y prueba, lo que refleja estabilidad. Además, el R^2 es de alrededor de 0.85, indicando que el modelo explica el 85% de la variabilidad de los datos. En conjunto, el modelo tiene un buen poder predictivo y un error promedio moderado.

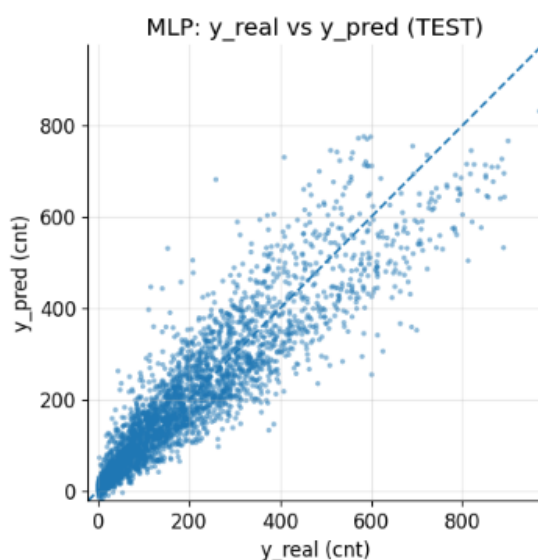


Imagen 17. MLP (Regressor): y_{real} vs y_{pred}

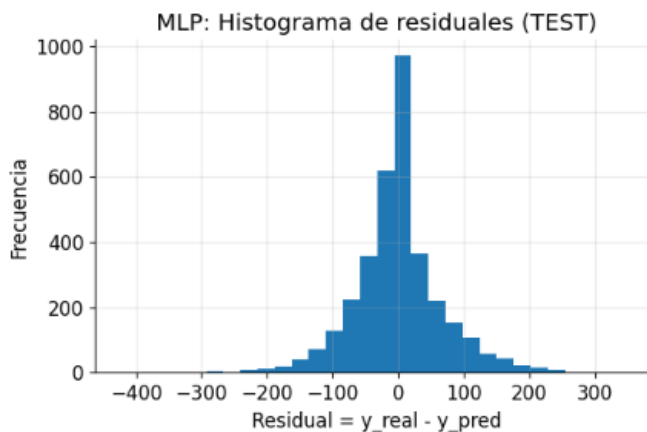


Imagen 18. MLP (Regressor): Histograma de residuales (TEST)

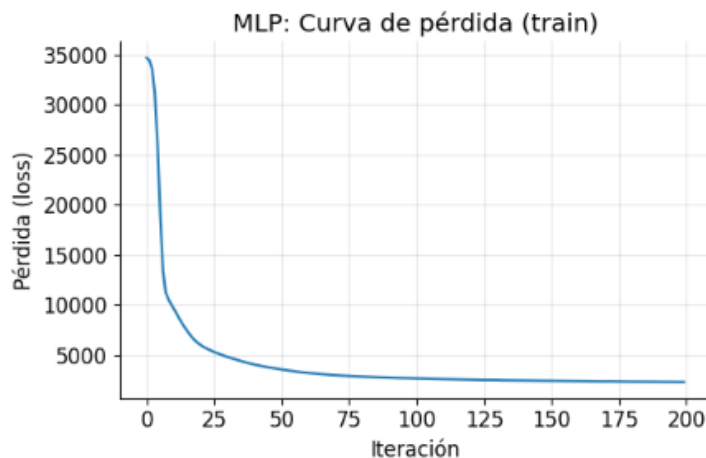


Imagen 19. MLP (Regressor): Curva de pérdida (train)

Gráfico y_real vs y_pred

== MLP (MEJOR MODELO) ==

Train -> MAE: 46.816 | RMSE: 68.336 | R2: 0.859

Test -> MAE: 46.512 | RMSE: 68.550 | R2: 0.852

Este modelo es ligeramente mejor que el anterior y mantiene un rendimiento sólido y equilibrado, con un comportamiento muy similar entre entrenamiento y prueba, lo que indica buena generalización y ausencia de sobreajuste. El MAE se encuentra entre 46.5 y 46.8, lo que implica que las predicciones fallan en promedio por unas 46–47 unidades. El RMSE está entre 68.3 y 68.6, mostrando gran estabilidad entre train y test, lo que sugiere pocos errores grandes. Además, el R^2 se sitúa entre 0.85 y 0.86, indicando que el modelo explica alrededor del 85% de la variabilidad de los datos. En conjunto, el modelo es estable, no presenta overfitting y tiene un

alto poder predictivo. Mejora ligeramente respecto al anterior, ya que reduce un poco el MAE y el RMSE en prueba, manteniendo prácticamente el mismo R^2 .

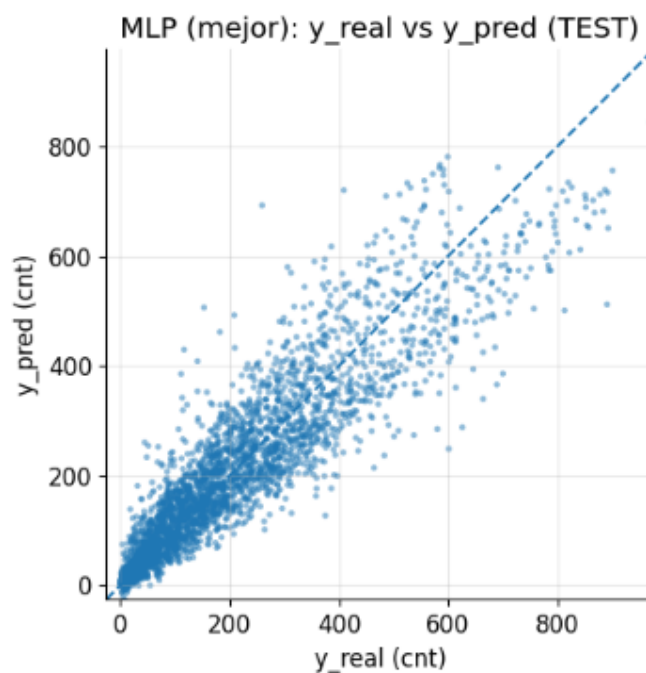


Imagen 20. MLP (Mejor): y_real vs y_pred

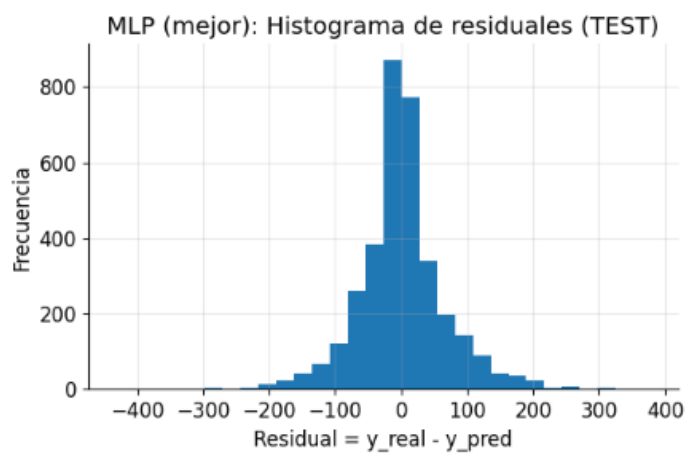


Imagen 21. MLP (Mejor): Histograma de residuales (TEST)

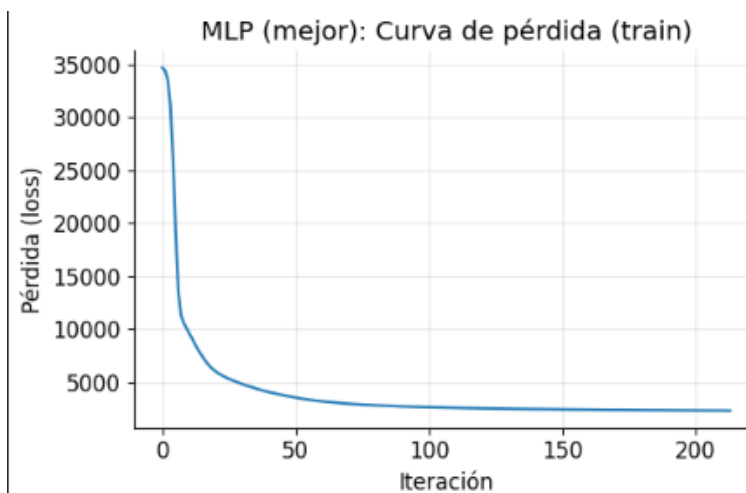


Imagen 22. MLP (Mejor): Curva de pérdida (train)

Preguntas Fase 3A:

- **Diferenciación de Métricas:** Explique la diferencia técnica entre el Error Absoluto Medio (MAE) y la Raíz del Error Cuadrático Medio (RMSE). ¿Cuál de las dos penaliza con mayor severidad los errores grandes (outliers) en las ventas y por qué?

Rta: El MAE (Error Absoluto Medio) mide el promedio de los errores en valor absoluto, es decir, trata todos los errores por igual sin importar su magnitud. Por otro lado, el RMSE (Raíz del Error Cuadrático Medio) eleva los errores al cuadrado antes de promediarlos, lo que hace que los errores grandes tengan un impacto mucho mayor.

Por esta razón, el RMSE penaliza más severamente los outliers, ya que un error grande al cuadrado se vuelve significativamente mayor. El **RMSE es más sensible a errores grandes**, mientras que el **MAE es más robusto**.

- **Diagnóstico de Desempeño:** Si el R^2 en el conjunto de entrenamiento es significativamente mayor que en el de prueba, ¿qué fenómeno está ocurriendo? Proponga una solución basada en lo aprendido en la Fase 1 del taller.

Rta: Si el R^2 en entrenamiento es mucho mayor que en test está ocurriendo sobreajuste (overfitting), pues el modelo aprende demasiado bien los datos de entrenamiento, pero no generaliza. Una solución de acuerdo a la fase 1 puede ser aplicar una regularización de Ridge aumentando el alpha y simplificando la red

- **Interpretación de Residuos:** Al observar la gráfica de y real vs. y predicho, ¿se identifica algún patrón de error sistemático (ej., subestimación en ventas altas)?

Rta: Si en la gráfica de y_{real} vs y_{pred} , se puede evidenciar una subestimación en valores altos pues el modelo no captura bien valores grandes y en la curva de perdida (Train) la forma tiene una relación no lineal no completamente aprendida, esto indica que el modelo aún puede mejorar en capturar patrones complejos.

Preguntas Fase 3B:

Consistencia de Parámetros: Compare el valor óptimo de alpha encontrado por el GridSearch con lo que analizó en la sección de Regresión Ridge. ¿Existe una relación entre la complejidad de la red y la necesidad de regularización?

Rta: El parámetro *alpha* en MLP cumple un rol similar al que tiene en Ridge, ya que actúa como un término de regularización que penaliza los pesos grandes. Esto ayuda a reducir el sobreajuste, evitando que el modelo se ajuste demasiado a los datos de entrenamiento y mejorando su capacidad de generalización.

Existe una relación clave entre la complejidad de la red y el valor adecuado de *alpha*. Las redes más complejas, con mayor número de neuronas o capas, suelen necesitar una regularización más fuerte, es decir, un valor de *alpha* más alto. En cambio, las redes más simples requieren menor regularización, por lo que un *alpha* más bajo suele ser suficiente.

Si en un proceso de *GridSearch* se selecciona un valor alto de *alpha*, esto indica que el modelo necesitaba un mayor control de su complejidad. En otras palabras, la regularización fue necesaria para evitar que la red se volviera demasiado flexible y propensa al sobreajuste.

Early Stopping: Explique cómo este mecanismo ayuda a optimizar el tiempo de cómputo y a prevenir el sobreajuste, basado en sus resultados.

Rta: El early stopping detiene el entrenamiento cuando el error de validación deja de mejorar. Es una técnica ampliamente utilizada para controlar el proceso de aprendizaje y evitar que el modelo continúe ajustándose cuando ya no obtiene mejoras reales en su desempeño. Entre sus principales beneficios se encuentran la prevención del sobreajuste, ya que evita que el modelo aprenda demasiado de los datos de entrenamiento, la reducción del tiempo de cómputo al detener el entrenamiento antes de alcanzar el número máximo de iteraciones, y la eliminación de iteraciones innecesarias que no aportan mejoras al modelo. En el modelo, es probable que no se alcance el valor de `max_iter`, debido a que el entrenamiento se detiene anticipadamente cuando deja de observarse una mejora en el error de validación.

Conclusión de Negocio: Basándose en el mejor modelo encontrado, ¿cuál es el margen de error promedio que la empresa debería esperar al predecir sus ventas?

Rta: El MAE representa el error promedio en unidades reales (ventas). Para nuestro caso si el MAE es 1.5, significa que el modelo se equivoca en promedio en 1.5 unidades de ventas, lo cual permite tomar decisiones más informadas en inversión publicitaria.

== MLP (MEJOR MODELO — Random Search amplio) ==

Train -> MAE: 40.012 | RMSE: 58.921 | R2: 0.895

Test -> MAE: 43.540 | RMSE: 63.719 | R2: 0.872

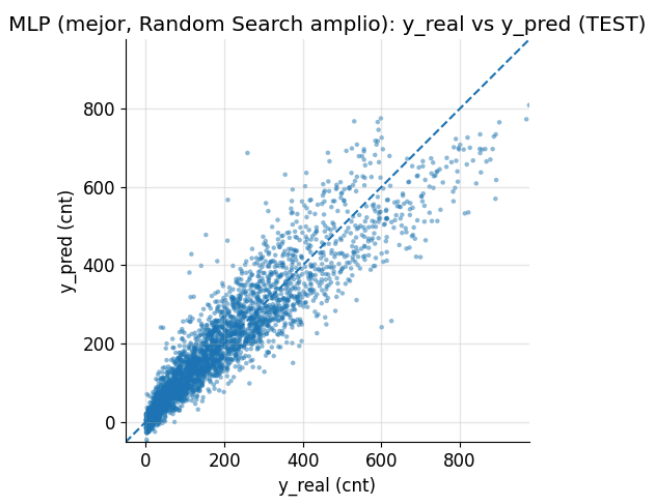


Imagen 23. MLP (Mejor, Random Search amplio): y_real vs y_pred (TEST)

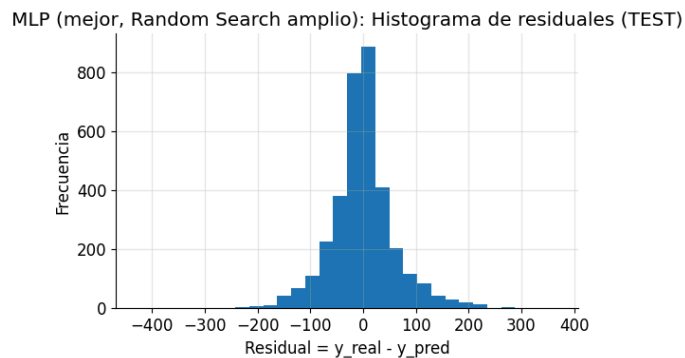


Imagen 24. MLP (Mejor, Random Search amplio): Histograma de residuales (TEST)

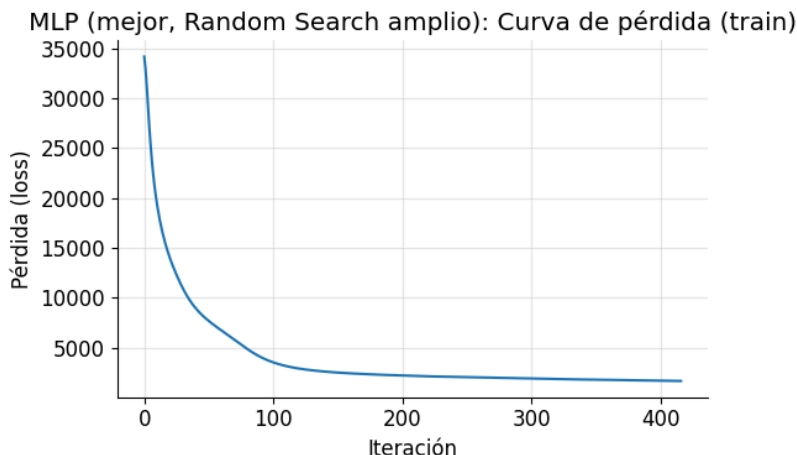


Imagen 25. MLP (Mejor, Random Search amplio): Curva de pérdida (train)

Las gráficas del mejor modelo MLP obtenido mediante Random Search evidencian un desempeño sólido y estable. La comparación entre valores reales y predichos muestra que la mayoría de las observaciones se ubican cerca de la línea ideal, indicando una buena capacidad predictiva del modelo. El histograma de residuales presenta una distribución centrada alrededor de cero, lo que sugiere ausencia de sesgo sistemático y errores generalmente pequeños. Finalmente, la curva de pérdida refleja una disminución progresiva del error hasta estabilizarse, indicando que el modelo logró converger adecuadamente durante el entrenamiento y aprendió de manera efectiva los patrones presentes en los datos.

4. Fase 4: Propagación hacia Adelante (Forward Propagation) Manual

Desarrollo Propagación hacia Adelante (Forward Propagation) Manual

Entrada normalizada

$$X = \begin{pmatrix} 0.5 \\ 0.3 \\ 0.2 \end{pmatrix}$$

Capa 1 (Oculta)

Pesos:

$$W^{(1)} = \begin{pmatrix} 0.2 & 0.5 & -0.1 \\ 0.8 & -0.3 & 0.4 \\ -0.5 & 0.1 & 0.2 \end{pmatrix}$$

Biases:

$$b^{(1)} = \begin{pmatrix} 0.1 \\ -0.2 \\ 0.0 \end{pmatrix}$$

Cálculo de $Z^{(1)} = W^{(1)}X + b^{(1)}$

Primera Neurona

$$(0.2)(0.5) + (0.5)(0.3) + (-0.1)(0.2) = 0.1 + 0.15 - 0.02 = 0.23$$

Sumando bias:

$$0.23 + 0.1 = 0.33$$

Segunda neurona:

$$(0.8)(0.5) + (-0.3)(0.3) + (0.4)(0.2) = 0.4 + 0.09 - 0.08 = 0.39$$

Sumando bias:

$$0.39 + 0.2 = 0.19$$

Tercera neurona:

$$(-0.5)(0.5) + (0.1)(0.3) + (0.2)(0.2) = -0.25 + 0.03 - 0.04 = -0.18$$

Sumando bias:

$$-0.18 + 0 = -0.18$$

$$Z^{(1)} = \begin{pmatrix} 0.33 \\ 0.19 \\ -0.18 \end{pmatrix}$$

Activación Sigmoide

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Primera neurona:

$$\sigma(1) = 0.5818$$

Segunda neurona:

$$\sigma(2) = 0.5474$$

Tercera neurona:

$$\sigma(3) = 0.4551$$

$$A^{(1)} = \begin{pmatrix} 0.582 \\ 0.547 \\ 0.455 \end{pmatrix}$$

Capa 2 (Salida)**Pesos:**

$$W^{(2)} = \begin{pmatrix} 0.5 & -0.4 & 0.1 \\ 0.2 & 0.8 & -0.5 \\ -0.1 & 0.3 & 0.7 \end{pmatrix}$$

Bias:

$$b^{(2)} = \begin{pmatrix} 0.1 \\ -0.2 \\ 0.1 \end{pmatrix}$$

$$\text{Cálculo de } Z^{(2)} = W^{(2)} A^1 + b^{(2)}$$

Primera Neurona

$$(0.5)(0.582) + (-0.4)(0.547) + (0.1)(0.455) = 0.291 + 0.19 - 0.0455 = 0.1175$$

Sumando bias:

$$0.1175 - 0.1 = 0.01175$$

Segunda neurona:

$$(0.2)(0.582) + (0.8)(0.547) + (-0.5)(0.455) = 0.1164 + 0.4376 - 0.2275 = 0.3265$$

Sumando bias:

$$0.3265 + 0.2 = 0.5265$$

Tercera neurona:

$$(-0.1)(0.582) + (0.3)(0.547) + (0.7)(0.455) = -0.0582 + 0.1641 + 0.3185 = 0.4244$$

Sumando bias:

$$0.4244 + 0.1 = 0.5244$$

$$Z^{(2)} = \begin{pmatrix} 0.0175 \\ 0.5265 \\ 0.5244 \end{pmatrix}$$

Softmax

$$P_i = \frac{e^{z_i}}{\sum e^{z_j}}$$

$$e^{0.0175} \approx 1.0177$$

$$e^{0.5265} \approx 1.693$$

$$e^{0.5244} \approx 1.689$$

Suma:

$$1.0177 + 1.693 + 1.689 = 4.3997$$

Probabilidades

$$P_1 = \frac{1.0177}{4.3997} = 0.231 \times 100\% = 23.1\%$$

$$P_2 = \frac{1.693}{4.3997} = 0.385 \times 100\% = 38.5\%$$

$$P_3 = \frac{1.689}{4.3997} = 0.384 \times 100\% = 38.4\%$$

La probabilidad mayor es $P_2 = 38.5\%$, por lo que la campaña se clasifica como Impacto Medio (C2) .

Preguntas Fase 4

- Interpretación de Resultados: Según las probabilidades obtenidas, ¿en qué categoría clasifica la red a esta campaña publicitaria?

Rta: Las probabilidades obtenidas para cada categoría fueron las siguientes: Bajo Impacto 23.1%, Impacto Medio 38.5% e Impacto Alto 38.4%.

Se observa que la clase con mayor probabilidad de Impacto Medio, aunque la diferencia con Impacto Alto es mínima. Esto indica que la campaña tiene una probabilidad intermedia-alta de generar un impacto significativo. En términos prácticos, el modelo considera que la campaña tiene un comportamiento competitivo entre impacto medio y alto, pero se inclina ligeramente hacia impacto medio como categoría final.

- Función de Activación: ¿Por qué utilizamos Softmax en la última capa en lugar de una función lineal o una simple sigmoide? Explique en términos de la suma de las probabilidades.

Rta: La función Softmax se utiliza en la capa de salida porque convierte los puntajes obtenidos (valores reales) en probabilidades. Una de sus principales características es que la suma de todas las probabilidades es igual a 1, lo que permite una interpretación clara en términos

probabilísticos. Además, Softmax es especialmente adecuada para problemas de clasificación multiclase, donde existen más de dos categorías posibles.

Si se utilizara una función lineal, no se obtendrían probabilidades, sino valores sin una interpretación directa. Por otro lado, la función sigmoide está diseñada principalmente para problemas de clasificación binaria, por lo que no sería la opción adecuada en este caso.

- Rol del Bias: ¿Qué sucedería con la capacidad de representación de la red si todos los vectores de sesgo (b) fueran cero?

Rta: El bias (sesgo) permite desplazar la función de activación, otorgando mayor flexibilidad al modelo. Si todos los valores de bias fueran cero, la red neuronal perdería capacidad de ajuste, ya que las fronteras de decisión quedarían limitadas al origen. Esto reduciría la capacidad de representación del modelo y afectaría su desempeño.

5. Fase 5: Evaluación de Modelos y Toma de Decisiones (Caso: Mantenimiento Predictivo)



Imagen 26. Distribución de Clases:(0 = No falla, 1 = Falla)

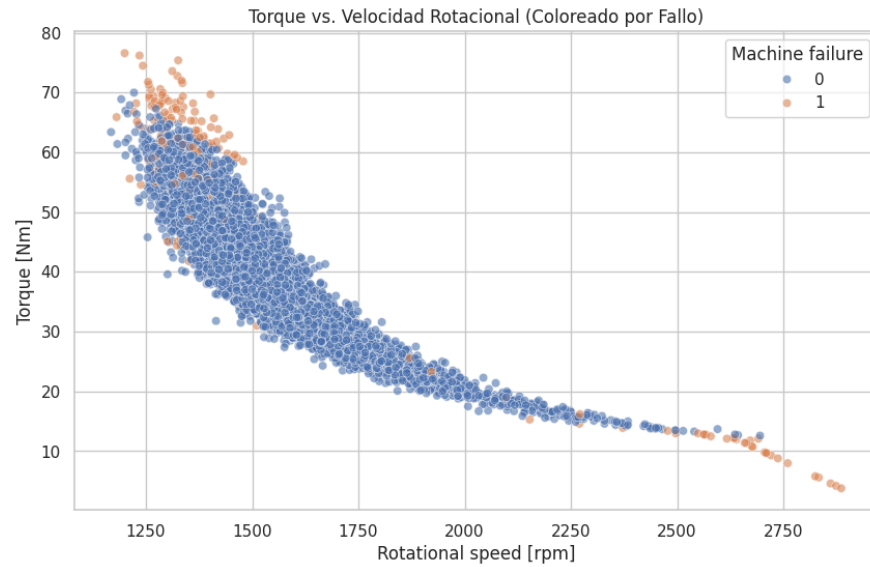


Imagen 27. Torque vs. Velocidad Rotacional: (Coloreado por Fallo)

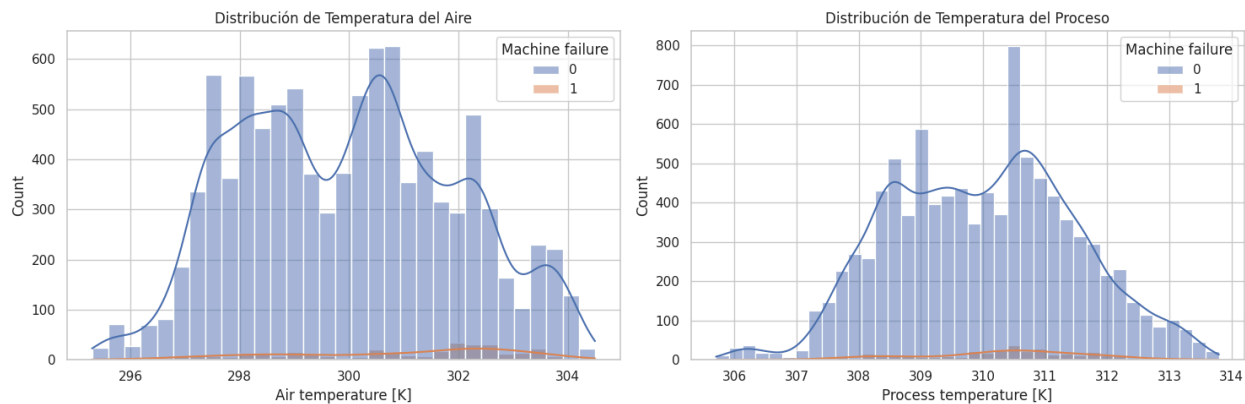


Imagen 27. Distribución de Temperatura del Aire y Distribución de Temperatura del Proceso.

Análisis de Pesos: Si el modelo indica que los errores en la Clase 1 (Fallo) pesarán aproximadamente 14.7 veces más que en la Clase 0, ¿qué intenta corregir el algoritmo al configurar el parámetro *class weight* durante el entrenamiento?

Rta: El parámetro `class_weight` se utiliza para compensar el desbalance de clases presente en el problema de mantenimiento predictivo, donde los fallos de máquina representan una proporción muy pequeña del total de observaciones. Al asignar un peso aproximadamente 14.7 veces mayor a la clase de fallos, el algoritmo penaliza más fuertemente los errores cometidos al no detectar una falla real.

Esto obliga al modelo a prestar mayor atención a la clase minoritaria, mejorando su capacidad para detectar eventos críticos. En términos prácticos, busca reducir los falsos negativos, que en este contexto representan el error más costoso para la empresa.

La trampa de la exactitud: explique por qué en este caso un *accuracy* del 95% podría considerarse un resultado deficiente para la gestión de mantenimiento si el modelo no logra detectar los fallos reales.

Rta: En un problema altamente desbalanceado como este, una exactitud elevada puede resultar engañosa. Por ejemplo, si solo el 5% de los casos corresponden a fallos, un modelo que siempre prediga “No Falla” podría alcanzar aproximadamente 95% de *accuracy* sin detectar ninguna falla real.

Por ello, aunque la exactitud es útil como referencia general, no es suficiente para evaluar el desempeño del modelo en este caso. Métricas como Recall, Precision y el costo económico de los errores ofrecen una evaluación mucho más representativa del valor real del modelo para el negocio.

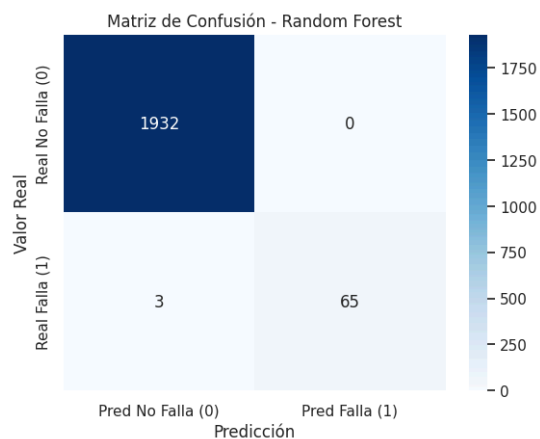


Imagen 27. Matriz de Confusión - Random Forest

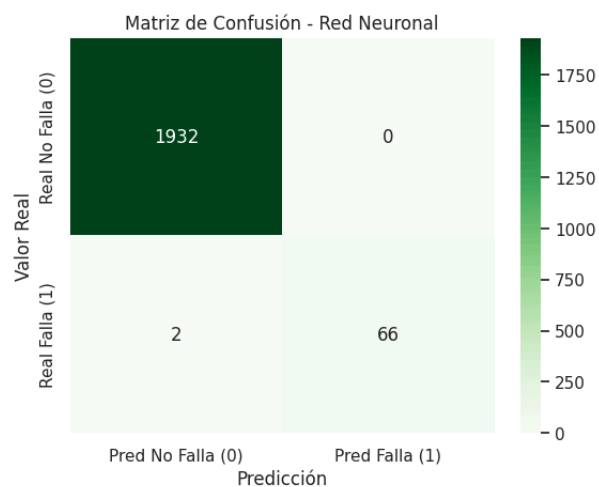


Imagen 27. Matriz de Confusión - Red Neuronal

Cálculo de métricas

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

$$Accuracy = \frac{66 + 1932}{2000} = 0.999$$

$$Accuracy = 99.9\%$$

Precisión

$$Precisión = \frac{TP}{TP+FP}$$

$$Precisión = \frac{66}{66+0} = 1$$

$$Precisión = 100\%$$

Cada vez que el modelo predice una falla, acierta.

$$Recall = \frac{TP}{TP+FN}$$

$$Precisión = \frac{66}{66+2} = 0.9706$$

$$Precisión = 97.06\%$$

El modelo detecta el 97.06% de las fallas reales.

Costo total del error

$$Costo = (FN \times 5000) + (FP \times 200)$$

$$Costo = (2 \times 5000) + (0 \times 200)$$

$$Costo = 10000$$

Costo Total Estimado = USD 10,000

Preguntas de reflexión y negocio – Fase 5

Trade-off entre *recall* y *precision*: En el contexto de evitar paradas de línea críticas, ¿preferiría usted un modelo con un *recall* del 98% pero una *precision* del 60%, o viceversa? Justifique su respuesta basándose en los costos calculados anteriormente.

Rta: En este contexto es preferible maximizar el Recall, incluso si ello implica sacrificar algo de Precision. Esto se debe a que no detectar una falla real (FN) tiene un costo extremadamente alto de USD 5,000, mientras que una falsa alarma solo cuesta USD 200.

Por lo tanto, un modelo con Recall del 98% y Precision del 60% podría ser más valioso para la empresa que uno con Recall menor, siempre que el costo de las falsas alarmas permanezca controlado.

Curva de aprendizaje: Observe la gráfica de *loss* (pérdida) y *accuracy* durante el entrenamiento. ¿Considera que 30 épocas son suficientes para que el modelo converja? ¿Qué sucede con el error de validación comparado con el de entrenamiento?

Rta: La curva de pérdida y accuracy observada durante el entrenamiento indica que 30 épocas fueron suficientes para que el modelo convergieron, ya que el error se estabilizó progresivamente sin mostrar mejoras significativas adicionales. Además, la cercanía entre las curvas de entrenamiento y validación sugiere que el modelo generaliza adecuadamente y no presenta sobreajuste severo.

Conclusión final del taller: Tras recorrer desde la regresión analítica hasta la clasificación con redes neuronales, ¿cómo puede el uso de métricas avanzadas (como el *recall*) transformar la rentabilidad de una empresa de manufactura?

Rta: La Red Neuronal resultó ser el mejor modelo para el problema de mantenimiento predictivo, superando ligeramente al Random Forest en capacidad de detección de fallos.

Aunque ambos modelos presentan una precisión perfecta del 100%, la Red Neuronal logra un Recall superior, detectando un mayor porcentaje de fallas reales y reduciendo el costo total de error en USD 5,000 frente al modelo alternativo.

Desde una perspectiva de negocio, este resultado demuestra que pequeñas mejoras en métricas como Recall pueden traducirse en ahorros económicos significativos, validando la importancia de seleccionar métricas alineadas con el costo real de los errores operativos.

Conclusión

A lo largo del taller se aplicaron múltiples técnicas de modelado estadístico y aprendizaje automático para resolver problemas de regresión, optimización y clasificación, permitiendo comprender la evolución desde modelos lineales tradicionales hasta arquitecturas avanzadas de redes neuronales.

Inicialmente, la regresión lineal múltiple permitió identificar relaciones significativas entre la inversión publicitaria y las ventas, mientras que la regularización Ridge mostró cómo controlar la complejidad del modelo y reducir el riesgo de sobreajuste. Posteriormente, el descenso de gradiente evidenció la importancia de la optimización numérica y de la correcta selección de hiperparámetros como la tasa de aprendizaje.

En fases posteriores, la implementación de redes neuronales multicapa permitió capturar relaciones no lineales complejas, mejorando la capacidad predictiva respecto a modelos lineales. Finalmente, en el contexto de mantenimiento predictivo, se demostró cómo una adecuada evaluación de métricas como Recall y costo económico permite traducir el desempeño técnico de un modelo en decisiones empresariales de alto impacto.

En conjunto, el taller evidencia que la combinación de técnicas de modelado, regularización, optimización y evaluación estratégica constituye una herramienta poderosa para transformar datos en decisiones de negocio más precisas, eficientes y rentables.

Referencias

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer.